

MEIC-MEIM - COMPUTAÇÃO DISTRIBUÍDA (exame época normal realizado em 28Jan2021 às 19h)

I PARTE (45 min)

Question 1

Not answered
Marked out of 1.00

Flag question

Edit question

[CDG10] Em relação às falácias da computação distribuída “A latência é Zero” e “A largura de Banda é Infinita”, responda se as seguintes afirmações são Verdadeiras ou Falsas:

- ☐ Latência e largura de banda são a mesma coisa pelo que não podem ser uma 0 e a outra infinita;
- ☐ Latência e Largura de Banda estão relacionados mas têm significados diferentes: Latência é o tempo de transferir dados entre dois pontos e Largura de Banda é a quantidade de dados por unidade de tempo, possível de transmitir entre dois pontos;
- ☐ Latência é o tempo (*timeout*) de espera pela resposta de uma invocação remota de um serviço;
- ☐ A Latência deve ser um fator a ter em conta no desenho de aplicações distribuídas, nomeadamente quando usam serviços em centros de dados geograficamente remotos (por exemplo na Cloud).

Question 2

Not answered
Marked out of 1.00

Flag question

Edit question

[CDG13] No desenho de aplicações distribuídas, é muito importante considerar o teorema CAP (*Consistency, Availability, Partitions*). Responda se as seguintes afirmações são Verdadeiras ou Falsas:

- ☐ Para manter disponibilidade (*Availability*) é conveniente suportar partições (*Partitions*), implicando a redução do nível de consistência (*Consistency*);
- ☐ O teorema CAP define que as três propriedades (*Consistency, Availability, Partitions*) têm de ser sempre garantidas a 100% numa aplicação distribuída;
- ☐ Num sistema de armazenamento de dados para aumentar a disponibilidade (*Availability*) criam-se réplicas em múltiplos nós (*Partitions*), permitindo que mesmo que alguns desses nós falhem os dados continuam a estar disponíveis. No entanto, nesses cenários não é possível garantir uma consistência (*Consistency*) total dos dados;
- ☐ Qualquer aplicação distribuída é por definição organizada em múltiplas partições (*Partitions*) (por exemplo várias máquinas virtuais) garantindo a disponibilidade (*Availability*) e acesso a dados de forma totalmente consistente (*Consistency*), mesmo na presença de falhas.

Question 3

Not answered
Marked out of 1.00

Flag question

Edit question

[CDG16] Qual o significado de afirmar que tanto em Java RMI como em gRPC se tem concorrência implícita no lado do servidor:

- ☐ Cada pedido é atendido por uma *Thread* do *runtime* não sendo necessário o programador criar uma *thread* para executar cada pedido;
- ☐ Porque podemos usar estruturas de dados concorrentes, por exemplo *ConcurrentHashMap*;
- ☐ Porque se um pedido de um cliente demora aproximadamente 3 segundos, mesmo quando temos N clientes concorrentes, cada pedido continua dos N clientes também demora aproximadamente 3 seg.;
- ☐ Porque usando *callbacks* em Java RMI e *stream* de servidor em gRPC, o cliente pode receber múltiplas notificações assíncronas.

Question 4

Not answered
Marked out of 1.00

Flag question

Edit question

[CDG11] Relativamente ao desenvolvimento de aplicações distribuídas em gRPC

- ☐ Numa operação com *stream* de cliente, o cliente tem a garantia que as mensagens chegam ao servidor pela ordem com que foram enviadas;
- ☐ A assinatura do método *onNext* da interface *StreamObserver* tem um número variável de parâmetros em função do contrato *protobuf*;
- ☐ Dado a definição de um serviço em *protobuf*, o servidor implementa métodos diferentes para executar chamadas via *stubs* bloqueantes ou *stubs* não bloqueantes.
- ☐ A utilização de *stubs*, para encapsular a chamada a serviços remotos, garante totalmente a característica de transparência à localização do serviço.

Question 5 Not answered Marked out of 1.00 Flag question Edit question	[CDG18] Ao desenvolver aplicações de computação distribuída o desacoplamento (<i>loosely coupling</i>) entre as partes é um o fator muito importante. Assim para ter desacoplamento numa aplicação cliente/servidor deve garantir-se: ☐ Usar no servidor diferentes <i>threads</i> para atender os pedidos; ☐ Definir um contrato (interface Java RMI ou contrato <i>protobuf</i> em gRPC) para separar as implementações do cliente e do servidor usando <i>stubs</i>; ☐ Numa chamada do cliente ao servidor a resposta (<i>reply</i>) só deve depender do pedido (<i>request</i>) e não depender de chamadas anteriores. ☐ Que o servidor é <i>stateless</i> por forma a facilitar escalabilidade horizontal (<i>scale out</i>);
Question 6 Not answered Marked out of 2.00 Flag question Edit question	[CDSA58] Os algoritmos de eleição e consenso são geralmente pesados devido ao grande número de mensagens envolvidas, principalmente quando o número de participantes é elevado e variável ao longo do tempo. Apresente as vantagens de implementar esses algoritmos usando um <i>middleware</i> de comunicação por grupos, com <i>multicast</i> fiável e conceito de <i>membership</i>, por exemplo o <i>Spread toolkit</i>.
Question 7 Not yet answered Marked out of 2.00 Flag question Edit question	[CDSA55] Considere um sistema distribuído, constituído por 3 participantes (p0, p1, p2), que utiliza comunicação por <i>multicast</i> fiável e um mecanismo baseado em relógios vetoriais que garante ordenação causal de mensagens. Quando p0 envia uma mensagem com o vetor [7, 1, 4], justifique porque acontece o seguinte: a) Em p1 a mensagem de p0 é retida (<i>Hold</i>) porque o vetor local é p1=[5, 1, 4]; b) Em p2 a mensagem de p0 é retida (<i>Hold</i>) porque o vetor local é p2=[6, 0, 5]; Está alterada e correta pois como se lembram existia uma gralha que levou à anulação da alínea b)
Question 8 Not answered Marked out of 2.00 Flag question Edit question	[CDSA50] Considere uma aplicação distribuída de processamento intensivo, baseada no modelo cliente/servidor, em que os múltiplos pedidos de múltiplos clientes exigem um elevado tempo de processamento. Justifique porque há vantagens de usar como intermediário, entre cliente e servidor, uma plataforma MOM (<i>Message Oriented Middleware</i>) ou <i>Publish/Subscribe</i> com suporte de filas de mensagens.

II PARTE (75 min)

Question 1

Not yet answered

Marked out of 3.00

Flag question

Edit question

[CDAT39] Considere uma empresa municipal que pretende implementar em gRPC um sistema cliente/servidor de afixação de anúncios/avisos sobre a pandemia em painéis instalados pela cidade, que implementa o seguinte contrato:

```
service GestPaineis {
  rpc register(Info) returns (stream Aviso);
  rpc unregister(Info) returns (Void);
}

message Void { }
```

```
message Info {
  int32 number = 1;
  string local = 2;
}

message Aviso {
  int32 tipo = 1;
  string txt = 2;
}
```

Descreva o esboço das classes que implementam o sistema, tanto do lado da aplicação cliente (no painel) como da aplicação servidora, indicando os pressupostos ou aspetos relevantes, usando uma notação do tipo comentário de código.

Note que não é necessário escrever código do tipo *import packages* e *void static main*, mas unicamente as partes importantes do ponto de vista gRPC.



Question 2

Not yet answered

Marked out of 3.00

Flag question

Edit question

[CDAT39] Considere uma fábrica onde uma aplicação cliente/servidor em Java RMI permite controlar múltiplas máquinas (aplicação cliente). O servidor implementa a interface **MachineServer** para que uma máquina se possa registar passando um *callback* para receber ordens de controlo.

```
public interface MachineServer extends Remote {
  void registrar(int myNumber, ControlCallback cb) throws RemoteException;
}

public interface ControlCallback extends Remote {
  void controlMachine(String ctl) throws RemoteException;
}
```

Descreva o esboço das classes que implementam o sistema, tanto do lado da aplicação cliente (*machine*) como da aplicação servidora, indicando os pressupostos ou aspetos relevantes, usando uma notação do tipo comentário de código.

Note que não é necessário escrever código do tipo *import packages* e *void static main*, mas unicamente as partes importantes do ponto de vista RMI.



Question 3

Not yet answered

Marked out of 3.00

Flag question

Edit question

[CDAT31] Considere um cenário em que existem centenas de bases de dados em milhares de organizações com dados que podem ser consultados para ajudar a perceber o comportamento e propagação do vírus COVID-19. Assim, pensar numa aplicação que faça uma mesma consulta a todas as bases de dados e depois realize o processamento de todas as respostas é impraticável, pois teria tempos de processamento e tempos de resposta ineficientes e pouco úteis face à necessidade de ter respostas. Assim foi convidado a propor uma aplicação distribuída de computação intensiva de larga escala que se execute em centenas de máquinas virtuais linux da plataforma *Google Cloud Platform* e uma componente cliente que se execute em computadores pessoais onde os utilizadores possam submeter consultas e recebam respostas rapidamente.

Proponha uma arquitetura dessa aplicação baseada nas plataformas de *middleware* estudadas indicando as interações e interfaces entre as partes envolvidas.

